Training-free lift baselines

The original LUDVIG and Occam's LGS code against our lift, on backroom and flightroom

Radiance FYP — semantic embedding

intellisense08 (RTX 2080, 8 GB), 13 October 2026

Abstract

We ran the authors' own code for two training-free lifting methods, LUDVIG (naver/ludvig, ICCV 2025) and Occam's LGS (insait-institute/OccamLGS), on our two scenes. We gave them exactly our Gaussians, our refined training cameras and our cached CLIP and DINOv2 teacher maps, and scored the resulting per-Gaussian tables with our unchanged query and evaluator. This isolates the lift from everything else. **LUDVIG's lift reproduces ours to numerical precision** (feature cosine 1.0000 on every Gaussian), so it scores identically. **Occam's centre-pixel lift is 6–18× faster** and agrees closely (CLIP cosine median 0.997). It finds the same number of objects with the multi-scale teachers (22/29) and one more with the scale-averaged teachers (21 vs 20/29). On 29 queries none of these differences is significant. The lift is not the bottleneck; the 2D teachers and the query are.

Contents

1 Key findings **1**

2 What was compared **2**
2.1 The three lifts . . . 2
2.2 Controlled design . . . 2
2.3 Adaptations needed to run the original code . . . 3

3 Verification: how close are the tables? **3**

4 Results on the Phase 5 sets **4**
4.1 Development set . . . 5

5 Floor sweep **7**

6 Cost **8**

7 Discussion **8**

8 Reproduction **9**

1 Key findings

1. **Our lift is LUDVIG's lift.** Using LUDVIG's CUDA kernel on the Inria rasteriser, every Gaussian's CLIP, DINO and per-scale CLIP row has cosine 1.0000 with ours on both scenes. The seen sets differ by at most one Gaussian. Hits, errors and peaks are identical on every query (§3, §4). Our gradient-based implementation is a faithful, independently verified implementation of LUDVIG eq. 3.

2. **Occam's LGS gives nearly the same tables, much faster.** It samples the feature only at the pixel under each Gaussian's projected centre. CLIP rows agree with ours to a median cosine of 0.997 and DINO rows to 0.98. It leaves 4.8 % (backroom) and 7.3 % (flightroom) of the Gaussians unseen, against 0.1–0.4 % for the all-pixel lift. It lifts the scale-averaged teachers in 28 s and 38 s, against our 181 s and 262 s.

3. **Localisation is unchanged within noise.** With the multi-scale teachers (A) all three methods hit 22/29. With the scale-averaged teachers (Old), Occam's hits 21/29 against 20/29 (one flightroom query). With DINO at query time (L-CD), Occam's loses 1–2 queries. Every difference is ≤ 2 of 29 queries, well inside the noise of this set.

4. **Consequence for the project.** The lift is settled. All-pixel and centre-pixel lifting both work, and the choice between them is about cost, not accuracy. Improvements have to come from the 2D teachers and the query: floor calibration, scale and region choice, recognition. Occam's speed makes it cheap to lift many more teacher maps, for example more scale groups or SAM levels, which is useful for `semantic/new`.

2 What was compared

2.1 The three lifts

All three give each Gaussian i a feature from the 2D teacher maps F_v of the training views v, weighted by the Gaussian's alpha-blending weight $w_i(v,p) = \alpha_i(v,p)\prod_{j<i}(1-\alpha_j(v,p))$.

Ours (`radiance_semantics/lift.py`, gsplat 1.0.0)
$f_i = \dfrac{\sum_{v,p} w_i(v,p)\,F_v(p)}{\sum_{v,p} w_i(v,p)}$ over *every pixel*. The numerator is the gradient of $\sum_p \langle \mathrm{render}(f), F_v\rangle$ at $f=0$ and the denominator the gradient of $\sum_p \mathrm{render}(\mathbf{1})$: two backward passes in 32-channel chunks (gsplat's backward limit).

LUDVIG (original code)
The same equation (paper eq. 3, $f = D^{-1}W^\top F$), computed by a dedicated CUDA kernel (`apply_weights`) inside the Inria rasteriser. It accumulates $w\,F$ and w in one forward pass.

Occam's LGS (original code)
The maximum-likelihood estimate simplified to the *centre pixel* (paper eq. 7): $f_i = \dfrac{\sum_v w_i^v\,F_v(\lfloor x_i'^v \rfloor)}{\sum_v w_i^v}$, where $x_i'^v$ is the projected centre and w_i^v the blend weight at that pixel, recorded by their gsplat 1.4.0 fork during an ordinary forward render. Gaussians never blended at their centre pixel are dropped ("filtering").

2.2 Controlled design

Only the lift changes. Figure 1 shows the data flow.

- **Same inputs.** `export_scene.py` (kitchen env) writes the trained splatfacto Gaussians in the Inria PLY layout, in checkpoint order, together with the 270 / 450 training views our lift uses: refined SO3×R3 poses, pose outliers dropped, centred principal points. Both runners read our cached teacher grids and upsample them to the 960 px render exactly as our lift does (bilinear, `align_corners=False`).
- **Original code.** Each method runs in its own conda env (`ludvig`: torch 2.4.0; `occamlgs`: torch 2.2.2 with their gsplat fork; both cu118 built for sm_75), never in kitchen. The runners call the papers' functions: `GaussianModel.apply_weights`; `gaussian_renderer.render` plus `accumulate_gaussian_feature_per_view`.

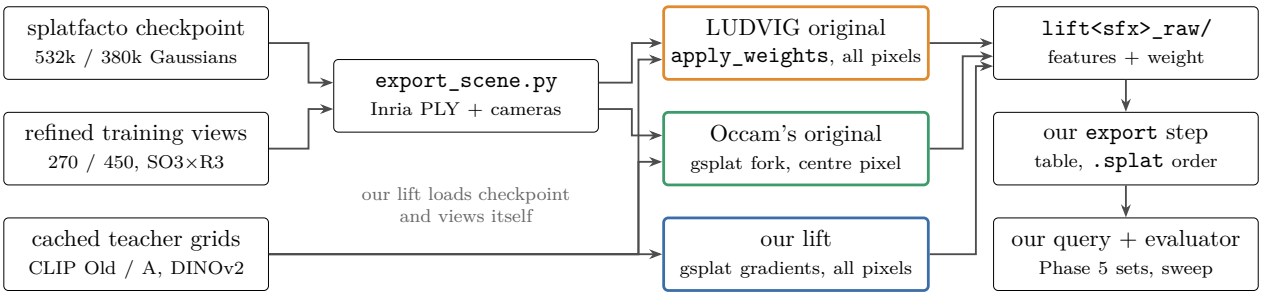


Figure 1: Controlled comparison. Gaussians, cameras and teacher maps are shared. Only the lift differs (blue: ours; orange: LUDVIG; green: Occam's LGS, both the authors' own code). Table building, query and evaluation are ours and unchanged.

- **Same scoring.** The runners write `lift<suffix>_raw/` in our lift step's layout. Our own `export` step builds the table in `.splat` order, and our unchanged evaluator scores it with the Phase 1 query settings (floor 0.55, $\tau = 0.55 + 0.5(\text{peak} - 0.55)$, 0.1 m voxels, $\geq$15 Gaussians).
- **Teachers.** Old = OpenCLIP ViT-B/16 7-scale pyramid, scales averaged (`d7a30d7264`). A = the same crops kept in three scale groups, the group chosen per query (`9c437cce68`, our default since 8 Oct). DINOv2 ViT-S/14 for the DINO rows.
- **Query sets.** The Phase 5 sets frozen on 6 Oct (backroom amended 7 Oct): 13 + 16 = 29 positives and 3 + 3 = 6 absent objects. They were not used to tune anything here. The Phase 1 development set (5 backroom queries) served as a first check.

2.3 Adaptations needed to run the original code

- **No pruning.** LUDVIG optionally keeps the top-k Gaussians by total weight, and Occam's removes never-activated Gaussians. We keep every row and give dropped Gaussians weight 0. To the query this is the same thing (it ignores unseen rows), and table rows must stay aligned with the `.splat` file.
- **LUDVIG's weight count.** The kernel adds αT to the weight counter once per *channel* (inside its channel loop), so its normaliser is $C \cdot \sum w$. In LUDVIG's own pipeline this scales every feature by $1/C$, which normalisation removes. We divide it out to store the true blend weight. The kernel is compiled for fixed channel counts, so our 384-d DINO is lifted in chunks of 256 + 100 + 20 + 3 + 3 + 2.
- **Occam's teacher mask.** Occam's code multiplies by LangSplat's validity mask. Our teacher maps cover every pixel, so the mask is all ones.
- **Rasteriser differences** (inherent to the originals, kept): the Inria rasteriser culls Gaussians within 0.2 m of the camera and clamps $\alpha \leq 0.99$; gsplat uses its own near plane. Both assume a centred principal point, which holds exactly on our scenes. Lens distortion is ignored by all three methods alike.

3 Verification: how close are the tables?

`table_agreement.py` compares every Gaussian seen by both tables. Rows are unit length, so the cosine is the dot product. Table 1 and Figure 2 give the results.

Reading. LUDVIG's kernel and our gradient trick compute the same sum, so the tables agree to float precision. Occam's rows are a single-pixel sample per view instead of a footprint average, so they differ slightly. CLIP differs least, because our CLIP teacher grid ($\approx 71 \times 40$ cells) is smooth at Gaussian scale and the centre pixel sees nearly the footprint's average. DINO differs more: its

Table 1: Per-Gaussian agreement with our lift (Gaussians seen by both). Median cosine, 1st-percentile cosine, and the share of rows with cosine > 0.99.

		backroom			flightroom		
Pair	Feature	median	p1	> 0.99 (%)	median	p1	> 0.99 (%)
Ours vs LUDVIG (Old)	CLIP	1.0000	1.000	100.0	1.0000	1.000	100.0
	DINO	1.0000	1.000	100.0	1.0000	1.000	100.0
Ours vs Occam (Old)	CLIP	0.9973	0.939	83.4	0.9970	0.935	79.6
	DINO	0.9809	0.624	31.2	0.9801	0.640	31.0
Ours vs LUDVIG (A)	CLIP	1.0000	1.000	100.0	1.0000	1.000	100.0
	DINO	1.0000	1.000	100.0	1.0000	1.000	100.0
Ours vs Occam (A)	CLIP	0.9974	0.942	84.3	0.9971	0.939	80.9
	DINO	0.9809	0.624	31.2	0.9801	0.640	31.0

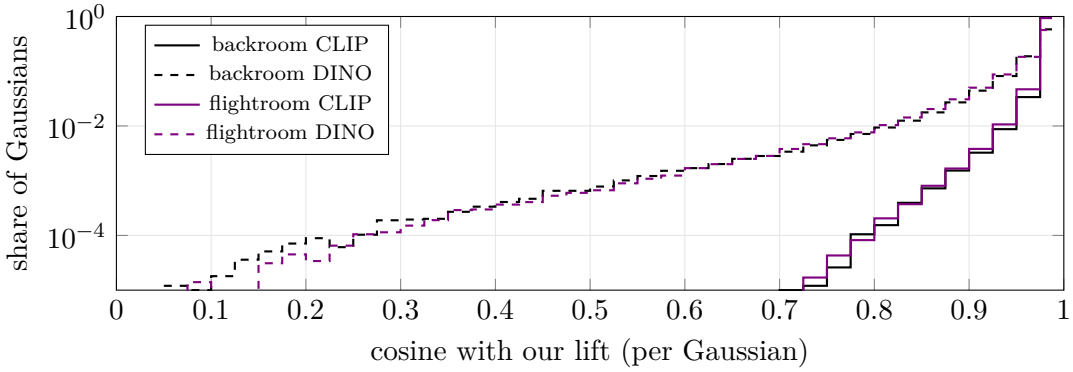


Figure 2: Occam's LGS vs our lift, Old teachers: distribution of per-Gaussian cosine (log scale). LUDVIG is not shown: every one of its rows has cosine 1.0000 with ours.

grid is similarly coarse, but neighbouring DINO cells differ more. A 3-view spot check before the full runs showed the same pattern (CLIP p1 0.958, DINO p1 0.75), and every Gaussian Occam's activated was also seen by our lift.

4 Results on the Phase 5 sets

Reading.

- **LUDVIG = ours, query for query.** The two lifts agree on all 29 positives and 6 absent objects, in both query modes and with both teacher sets. Median errors are identical, and the 90th-percentile errors differ by at most 1 mm. Only the query times differ (0.53–3.2 s), which is CPU noise: the tables hold the same numbers.
- **Occam's LGS: ±1–2 queries, in both directions.** With CLIP alone (L-C), Occam's Old table gains the flightroom *armchair* (21 vs 20/29). Occam's A table swaps *office chair* (lost) for *camera tripod* (gained, the first hit on this query by any lift or teacher), so it ties at 22/29. With DINO at query time (L-CD), Occam's loses *water bottle* (Old, 19/29) and *swivel chair*, *upholstered chair* and *water bottle* while gaining *water jug* (A, 20/29). Its DINO rows differ most from the all-pixel lift (§3), and L-CD's diffusion and split use exactly those rows.
- **The teachers move as many queries as the lift, or more.** Going from Old to A changes 3–4 L-C outcomes for every lift: *hose reel* is lost, *upholstered chair* and *water jug* are gained, and so is *armchair* for ours and LUDVIG's (Occam's already had it with Old). Going from our lift to Occam's on the same teachers changes one L-C outcome (Old) or two (A), and one (Old) or four (A) L-CD outcomes. The scale-group choice of A recovers the same occluded chairs and the water jug whichever lift produced the table.

Table 2: Phase 5 evaluation, top-1 hits per scene and combined (29 positives), median localisation error of the returned candidates, absent objects that returned a candidate (of 3 per scene), and AUROC of peak relevancy (positives vs absent). Query settings unchanged from Phase 1.

			backroom (13 + 3)				flightroom (16 + 3)			
Lift	Teach.	Total	hits	err (m)	FP	AUROC	hits	err (m)	FP	AUROC
L-C (CLIP relevancy only)										
Ours (gsplat grad.)	Old	**20**/29	10/13	0.19	1/3	0.85	10/16	0.22	1/3	0.90
LUDVIG (original)	Old	**20**/29	10/13	0.19	1/3	0.85	10/16	0.22	1/3	0.90
Occam's LGS (original)	Old	**21**/29	10/13	0.18	1/3	0.85	11/16	0.18	1/3	0.92
Ours (gsplat grad.)	A	**22**/29	9/13	0.21	1/3	0.87	13/16	0.25	1/3	0.90
LUDVIG (original)	A	**22**/29	9/13	0.21	1/3	0.87	13/16	0.25	1/3	0.90
Occam's LGS (original)	A	**22**/29	9/13	0.26	2/3	0.87	13/16	0.18	1/3	0.92
L-CD (+ DINO diffusion and split at query time)										
Ours (gsplat grad.)	Old	**20**/29	10/13	0.19	1/3	0.85	10/16	0.21	0/3	0.90
LUDVIG (original)	Old	**20**/29	10/13	0.19	1/3	0.85	10/16	0.21	0/3	0.90
Occam's LGS (original)	Old	**19**/29	10/13	0.15	1/3	0.87	9/16	0.20	0/3	0.92
Ours (gsplat grad.)	A	**22**/29	10/13	0.25	1/3	0.87	12/16	0.25	1/3	0.92
LUDVIG (original)	A	**22**/29	10/13	0.25	1/3	0.87	12/16	0.25	1/3	0.92
Occam's LGS (original)	A	**20**/29	9/13	0.26	1/3	0.85	11/16	0.23	1/3	0.92

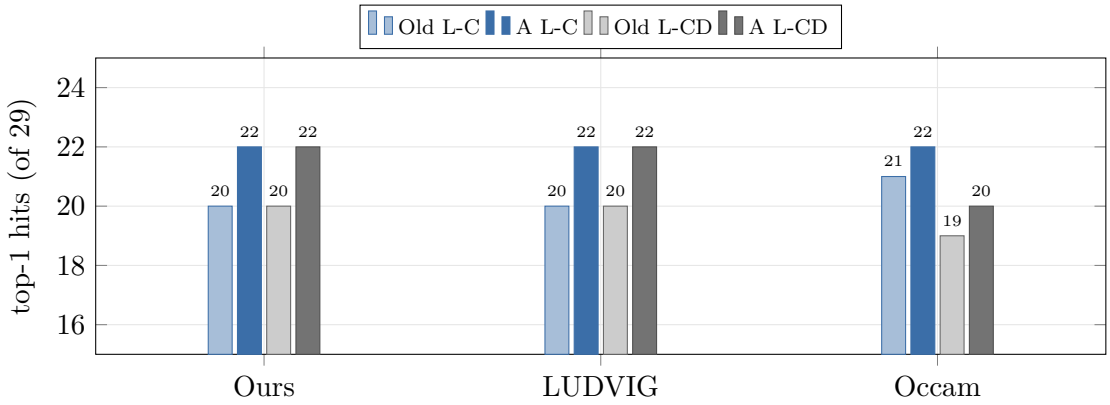


Figure 3: Combined top-1 hits on both scenes, by lift method, teachers and query mode (L-C: CLIP relevancy; L-CD: plus DINO diffusion and split).

- **Localisation.** Occam's median errors are equal or smaller with the Old teachers (0.18 vs 0.19 m backroom, 0.18 vs 0.22 m flightroom) and mixed with A (0.26 vs 0.21 m backroom, 0.18 vs 0.25 m flightroom). Backroom A has two "bad position" misses with Occam's against one with ours.
- **Absent objects.** AUROC is equal or higher with Occam's (flightroom 0.917 vs 0.896). At floor 0.55, however, Occam's A table fires on both *bicycle* and *ceiling fan* in backroom (2/3 vs 1/3). With the Old teachers, sampling one pixel instead of averaging the footprint raises peaks slightly (dev-set peaks +0.003 to +0.018), absent objects included; with A the shifts are mixed (−0.010 to +0.012).

4.1 Development set

On the 5 backroom development queries (Phase 1 gate) our and LUDVIG's tables hit 5/5 with identical peaks, and Occam's hit 4/5 (Table 4). With the Old teachers Occam's *garden cart* candidate is a small cluster (81 Gaussians against our 94) whose centroid lands 0.99 m from the annotation. With A, *green foam mats* picks a mat-like region 2.9 m away. Both are threshold-edge cases of the cluster rules, not systematic failures. The frozen sets (above) are the evaluation.

Table 3: Per-query outcome, L-C (● hit, ○ miss). Queries in bold differ between methods.

	Old teachers			A teachers		
Query	Ours	LUDVIG	Occam	Ours	LUDVIG	Occam
backroom						
recycling bin	●	●	●	●	●	●
blue trash can	●	●	●	●	●	●
office chair	●	●	●	●	●	○
swivel chair	○	○	○	○	○	○
grandfather clock	●	●	●	●	●	●
clock	●	●	●	●	●	●
children's road play mat	●	●	●	●	●	●
camera tripod	○	○	○	○	○	●
purple foam mat	●	●	●	●	●	●
white bucket	○	○	○	○	○	○
glass door cabinet	●	●	●	●	●	●
hose reel	●	●	●	○	○	○
moving box	●	●	●	●	●	●
flightroom						
armchair	○	○	●	●	●	●
upholstered chair	○	○	○	●	●	●
whiteboard	●	●	●	●	●	●
computer monitor	●	●	●	●	●	●
keyboard	○	○	○	○	○	○
floor lamp	○	○	○	○	○	○
red cup	●	●	●	●	●	●
water bottle	●	●	●	●	●	●
water jug	○	○	○	●	●	●
round table	●	●	●	●	●	●
drone gate	○	○	○	○	○	○
PVC pipe frame	●	●	●	●	●	●
poster	●	●	●	●	●	●
garden cart	●	●	●	●	●	●
wheelbarrow	●	●	●	●	●	●
cardboard box	●	●	●	●	●	●
Total (of 29)	20	20	21	22	22	22

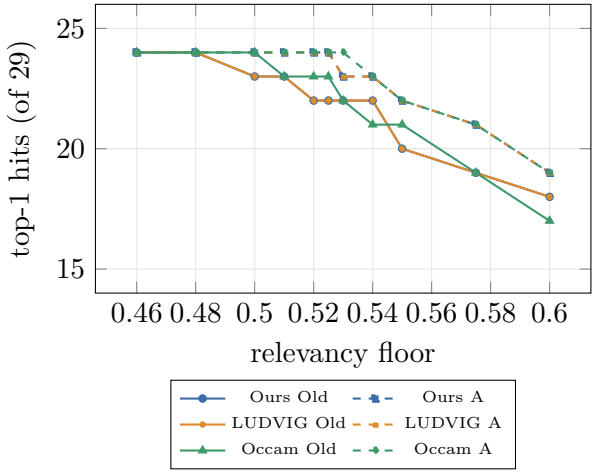
Table 4: Development set (5 backroom queries, `phase1_gate`): hits, median error and peak relevancy per query.

Lift, teachers	hits	err (m)	peak relevancy per query[†]
Ours, Old	5/5	0.22	0.789 0.627 0.719 0.590 0.696
LUDVIG, Old	5/5	0.22	0.789 0.627 0.719 0.590 0.696
Occam's, Old	4/5	0.31	0.792 0.631 0.737 0.600 0.704
Ours, A	5/5	0.38	0.821 0.681 0.789 0.683 0.734
LUDVIG, A	5/5	0.39	0.821 0.681 0.789 0.683 0.734
Occam's, A	4/5	0.36	0.823 0.683 0.787 0.673 0.746

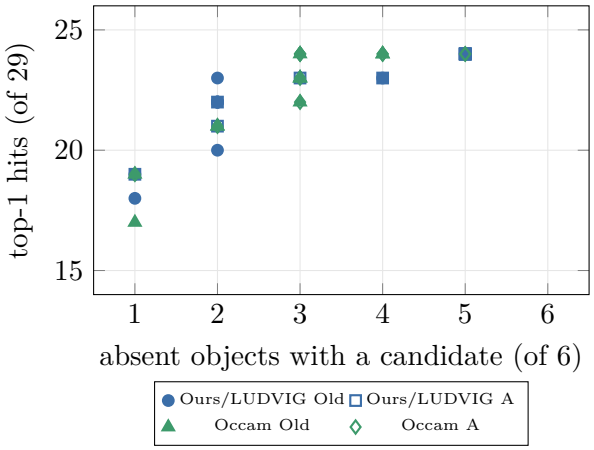
[†] red tool chest, shop vacuum, green foam mats, garden cart, whiteboard

5 Floor sweep

The relevancy floor (0.55) is the one query setting that matters most (SEMANTICS.md §13, §16). Sweeping it shows whether a lift shifts the relevancy scale, which would make a comparison at one fixed floor unfair. The sweep is post-hoc on the evaluation sets and is reported as such.



(a) Hits vs floor.



(b) Hits vs false positives (each point one floor).

Figure 4: Relevancy-floor sweep (floors 0.46–0.60), both scenes combined. LUDVIG's curves lie exactly on ours.

Reading. LUDVIG's curves coincide with ours at every floor, as expected from identical tables. Occam's curves cross ours. Its peaks are slightly higher, so at a given floor it keeps a few more candidates. When the methods are compared at an equal number of absent objects firing, which is the fair comparison:

Absent objects with a candidate (of 6)	1	2	3	4
Ours / LUDVIG, Old	19	**23**	23	23
Ours / LUDVIG, A	19	22	23	23
Occam's, Old	19	21	**24**	**24**
Occam's, A	19	21	**24**	**24**

(best hits over the floors 0.46–0.60 with at most that many absent objects firing). Occam's is ahead by one query at 3–4 false positives and behind by one or two at 2; at 1 false positive every method finds 19. The ceiling over all floors is 24/29 for every lift and teacher set, the same ceiling the 8 Oct sweep found for B2. As on 8 Oct, the same two absent objects keep firing whatever the lift: *ceiling fan* (backroom) at every floor for every table, and *recycling bin* (flightroom) up to floor 0.55 with the Old teachers and 0.575 with A. These floors are read off the evaluation set and are optimistic.

6 Cost

Table 5: Lift cost on the RTX 2080 (960 px renders): wall time of the lift step, peak GPU memory, and share of Gaussians seen (weight > 0). Teacher extraction (31–53 min) is shared by all and excluded.

		backroom (532k G., 270 views)			flightroom (380k G., 450 views)		
Lift	Teach.	time (s)	GPU (GB)	seen (%)	time (s)	GPU (GB)	seen (%)
Ours (gsplat grad.)	Old	181	7.4	99.9	262	2.4	99.6
LUDVIG (original)	Old	369	3.3	99.9	633	2.7	99.6
Occam's LGS (original)	Old	28	3.3	95.2	38	2.6	92.7
Ours (gsplat grad.)	A	1064	0.8	99.9	1364	0.8	99.6
LUDVIG (original)	A	784	3.3	99.9	1352	2.7	99.6
Occam's LGS (original)	A	124	3.3	95.2	74	2.6	92.7

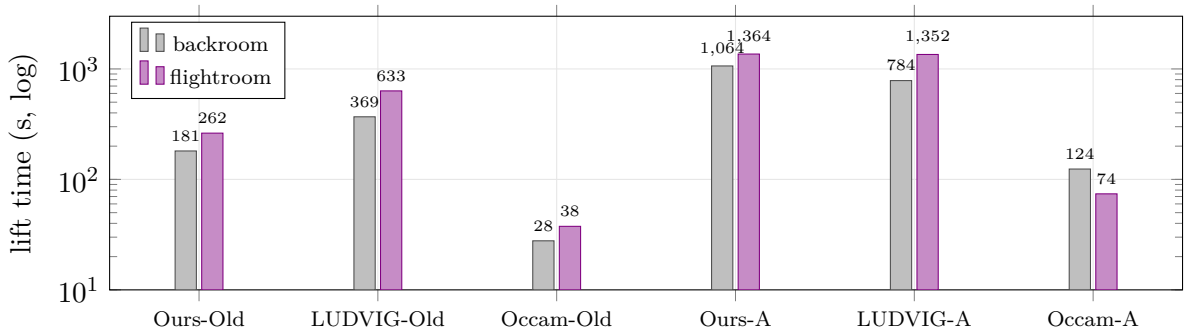


Figure 5: Lift wall time per scene (log scale).

Reading. Occam's centre-pixel lift is the cheapest by far: one forward render per view (no backward pass), with the feature gathered at one pixel per Gaussian. It lifts all 512 CLIP and 384 DINO channels in **28 s / 38 s** (Old) and **124 s / 74 s** (A, four maps), 6.5–18× faster than our gradient lift (181 s / 262 s and 1064 s / 1364 s). LUDVIG's kernel is the slowest for wide features (369 s / 633 s for Old). It processes every pixel's full channel vector per Gaussian, and its 512-channel template likely spills registers on sm_75. Our A lift stays under 1 GB of GPU memory because its accumulators live in host RAM; the two originals keep them on the GPU (2.7–3.4 GB), which still fits the 8 GB card. Coverage is the price of Occam's speed: 4.8 % and 7.3 % of the Gaussians are never blended at their own centre pixel and stay unseen (all-pixel lifts: 0.1 % and 0.4 %). These are mostly small or occluded Gaussians, which the query's opacity and cluster-size filters largely ignore anyway. Teacher extraction (31–53 min per scene) dominates every pipeline.

7 Discussion

Why the lifts barely matter here. The Phase 5 diagnostic (SEMANTICS.md §14) found that lifting into 3D loses almost nothing: relevancy near an annotated object is 0.007 lower in the table than in the 2D teacher grid. This experiment confirms it from the other side. Two quite different estimators, a footprint-weighted average over every pixel and a single-pixel sample per view, produce near-identical tables (CLIP cosine ≥ 0.997 in median) and the same localisation within ± 2 of 29 queries. The CLIP teacher grid is coarse ($\approx 71 \times 40$ cells per image), so at Gaussian scale it is almost constant across a footprint, and how the footprint is sampled hardly changes the result. The paper-level claims match this: Occam's LGS reports its estimator as the maximum-likelihood solution under negligible Gaussian overlap, and LUDVIG reports that further optimisation steps do not improve its features.

Statistical caveat. 29 positives and 6 absent objects over two scenes. A difference of one or two queries is not significant (exact sign test on the discordant queries, $p \geq 0.6$ for every pair). The development-set result (Occam's 4/5 vs 5/5) is one cluster of 81 Gaussians landing 0.99 m from the garden cart (Old) and one mat-like region chosen over the green foam mats (A), both near the edge of the query's cluster rules.

What this means for the next steps.

- **Keep the all-pixel lift as the reference.** It is LUDVIG's lift, now verified against the authors' code. It sees the most Gaussians and averages each one over its whole footprint.
- **Use centre-pixel lifting where speed matters.** At 30–120 s per scene it makes many more teacher maps affordable: more crop-scale groups, SAM levels (B2), or per-query re-lifts. It is a natural building block for `semantic/new`.
- **Put the effort into the 2D side and the query.** This is where queries are won or lost: the floor calibration (HANDOVER §2.1), per-query scale or region choice, recognition of thin objects (tripod, keyboard, floor lamp), and the two persistent "absent" detections.
- **Still to run, the faithful pipelines.** This report isolates the lift. The papers' full pipelines differ in more than the lift: LUDVIG's DINOv2 ViT-g features with registers, PCA and logistic-regression graph diffusion; Occam's SAM-based LangSplat feature maps. Those are the next comparisons on these branches. LUDVIG's diffusion is the part most likely to change outcomes, because our simplified L-CD diffusion changed no hit.

8 Reproduction

Code lives on the branches `semantic/LUDVIG` (`semantics/baselines/ludvig_lift.py`) and `semantic/LGS` (`semantics/baselines/occam_lift.py`). Both carry the shared `env.sh`, `install_*.sh` and `export_scene.py`. The original repositories are cloned, unmodified, in `~/Radiance/third_party/`.

```
bash semantics/baselines/install_ludvig.sh          # one-off: conda env ‘ludvig‘
bash semantics/baselines/install_occamlgs.sh        # one-off: conda env ‘occamlgs‘
source ~/Radiance/figs/figs_env.sh                  # kitchen
python semantics/baselines/export_scene.py --scene backroom
source semantics/baselines/env.sh ludvig
python semantics/baselines/ludvig_lift.py --scene backroom --teachers d7a30d7264 --suffix _ludvigpyr
source semantics/baselines/env.sh occamlgs
python semantics/baselines/occam_lift.py  --scene backroom --teachers 9c437cce68 --suffix _occamms
source ~/Radiance/figs/figs_env.sh
python figs/semantic_pipeline.py --scene backroom --backend lift --table-suffix _occamms \
       --clip-mode scales --only export
python -m radiance_semantics.evaluate --scene backroom --variants L-C@occamms L-CD@occamms
```

Raw results (evaluator JSONs, lift statistics, sweep, agreement, logs) are in `../raw/`, derived CSVs in `../csv/`. `../scripts/collect.py` and `make_tables.py` regenerate everything this report reads.